

Pokhara, Gandaki Province, Nepal

A

Project Report

On

NepMedAI: A Retrieval-Augmented Generation Based Medical Information Assistant for Nepali Language Users

For the partial fulfillment of the requirements for Micro-degree of AI/ML
Under Gandaki University

Submitted to

AI Community in Nepal (AICN)

Under the supervision of

Mr. Diwash Sapkota

Lecturer, Artificial Intelligence and Machine Learning Micro-degree

Submitted by

Binod Raj Pant

Sampurna Bhujel

Submission Date: 30/05/2026

May, 2026

ACKNOWLEDGEMENT

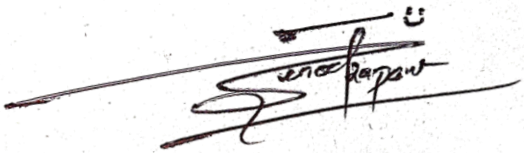
We are very grateful to the Artificial Intelligence community in Nepal (AICN), Gandaki University and National Innovation Center (NIC) for providing us an opportunity to work on a project as part of Artificial Intelligence and Machine Learning Micro-degree. We are delighted to express out deep sense of gratitude and gratefulness to our supervisor sir **Mr. Diwash Sapkota** for his invaluable guidance, encouragement, and even monitoring to spare time despite his busy schedule for project progress reviews.

We are likewise grateful to the faculty and coordinators of the AI/ML Microdegree Program at AI Community in Nepal (AICN) for creating an intellectually stimulating environment and for providing access to the resources necessary to undertake research of this nature.

Our appreciation also goes to the broader open-source community whose publicly available datasets, libraries, and research publications have formed the foundation upon which this project is built.

Finally, we thank our families and peers for their patience and moral support during this academic endeavour.

With Regards



Binod Raj Pant



Sampurna Bhujel

Date: 30/05/2026

STUDENT'S DECLARATION

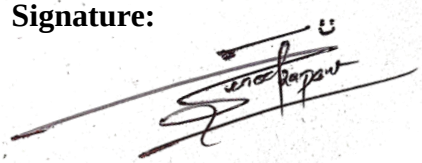
We, Binod Raj Pant and Sampurna Bhujel, hereby declare that this project proposal, titled "NepMedAI: A Retrieval-Augmented Generation Based Medical Information Assistant for Nepali Language Users," has been prepared independently and represents our original academic contribution. To the best of our knowledge, no portion of this work has been submitted to any other institution for the purpose of obtaining any qualification or award.

All external materials, including research papers, datasets, and technical references, have been duly acknowledged and cited in accordance with the citation standards specified by the program. Any resemblance to pre-existing projects or publications is purely unintentional.

We are aware that academic dishonesty, including plagiarism or misrepresentation of authorship, constitutes a serious violation of the program's academic integrity policy and may lead to disqualification from the program.

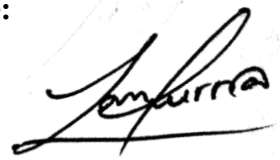
Student Name: Binod Raj Pant

Signature:



Student Name: Sampurna Bhujel

Signature:



Program: Artificial Intelligence and Machine Learning Micro-degree

Date: 30/05/2026

SUPERVISOR'S DECLARATION

This is to certify that the project report entitled **"NepMedAI: A Retrieval-Augmented Generation Based Medical Information Assistant for Nepali Language Users"** has been prepared by Binod Raj Pant and Sampurna Bhujel under my direct supervision and guidance.

The proposal has been reviewed for academic merit, structural compliance, and alignment with the objectives of the AI/ML Micro-degree Program. To the best of my assessment, the work satisfies the required academic standards and is approved for formal submission and evaluation.

_____

Mr. Diwash Sapkota
Project Supervisor,
Lecturer,
Artificial Intelligence and Machine Learning Micro-degree
AI Community In Nepal (AICN)
Pokhara, Gandaki Province, Nepal
Date: 30/05/2026

ABSTRACT

NepMedAI is a bilingual Retrieval-Augmented Generation (RAG) based medical information assistant developed to address the critical absence of verified, Nepali-language digital health resources in Nepal. The system was implemented in Python using LangChain as the orchestration framework, ChromaDB as a persistent local vector database, and LLaMA 3.1 8B Instant via the Groq API as the large language model. A bilingual sentence embedding model was used to encode both documents and user queries into dense vector representations, enabling semantic similarity-based retrieval.

The knowledge base was assembled from six publicly available, authoritative medical documents including WHO clinical handbooks, Government of Nepal health guidelines, and an internationally recognised medical encyclopaedia. These documents were loaded, split into 800-token chunks with a 150-token overlap using RecursiveCharacterTextSplitter, and persisted in ChromaDB. At inference time, the top-4 semantically similar chunks are retrieved for any given user query, injected into a structured PromptTemplate alongside the last three conversation turns, and passed to the LLM to generate a context-grounded response.

The user interface was built using Streamlit with a custom dark-themed CSS design, featuring bilingual suggestion chips, source attribution above each response, and a mandatory bilingual medical disclaimer. The system correctly matched response language to query language across Nepali-only, English-only, and mixed-language inputs. Five distinct error conditions were handled gracefully with bilingual error messages. The project was completed within a six-week development cycle and the full codebase is publicly available on GitHub.

Keywords: *Retrieval-Augmented Generation, Nepali NLP, Medical AI, Chatbot, LangChain, Vector Database, Healthcare Informatics*

LIST OF ABBREVIATIONS

AI : Artificial Intelligence

AIML : Artificial Intelligence and Machine Learning

API: Application Programming Interface

BERT: Bidirectional Encoder Representations from Transformers

DOC : Document Open ML

ERD : Entity Relationship Diagram

GU : Gandaki University

LLM: Large Language Model

NLP : Natural Language Processing

PDF : Portable Document Format

RAG : Retrieval-Augmented Generation

UI : User Interface

UML: Unified Modeling Language

TABLE OF FIGURES

Figure 5.9: Model Pipeline 8
Figure 7.1: System Architecture 11
Figure 7.2: Data Flow Diagram Level 0 11
Figure 7.3: Data Flow Diagram Level 1 12
Figure 7.4: Query Procesing FlowChart 12
Figure 9: Development Plan 12
Figure 15: Gantt Chart 12

LIST OF TABLES

Table 5.2: Documents Used 6
Table 6.1: Functional Requirements 9
Table 6.2: Non-Functional Requirements 9
Table 6.3: Technology Stack 10
Table 10.1: Testing Strategy 17

TABLE OF CONTENT

ACKNOWLEDGEMENT i
STUDENT'S DECLARATION ii
SUPERVISOR'S DECLARATION iii
ABSTRACT iv
LIST OF ABBREVIATIONS v
TABLE OF FIGURES vi
LIST OF TABLES vii
CHAPTER 1:INTRODUCTION 1
1.1 Background and Motivation 1
1.2 Project Scope 1
CHAPTER 2: PROBLEM STATEMENT 2
CHAPTER 3: OBJECTIVES 3
CHAPTER 4: LITERATURE REVIEW 4
4.1 Retrieval-Augmented Generation (Lewis et al., NeurIPS 2020) [1] 4
4.2 REALM: Retrieval-Augmented Language Model Pre-Training (Guu et al., ICML 2020) [2] 4
4.3 MedQA: Medical Board E am QA (Jin et al., Applied Sciences 2021) [3] 4
4.4 Large Language Models Encode Clinical Knowledge Med-PaLM (Singhal et al., Nature 2023) [4] 4
4.5 NepBERTa: Nepali Language Model (Timilsina et al., AACL-IJCNLP 2022) [5] 5
CHAPTER 5: METHODOLOGY 6
5.1 System Architecture 6
5.2 Document Ingestion and Pre-processing 6
5.3 Embedding and Vector Inde ing 7
5.4 Retrieval 7
5.5 Prompt Design 7
5.6 LLM Configuration 7
5.7 Conversation History Management 8
5.8 Error Handling 8
5.9 Model Pipeline Diagram 8
CHAPTER 6: REQUIREMENT DOCUMENT 9
6.1 Functional Requirements 9
6.2 Non-Functional Requirements 9
6.3 Technology Stack 9
CHAPTER 7: SYSTEM ANALYSIS AND DESIGN 11

7.1 System Architecture Overview 11
7.2 Data Flow Diagram Level 0 11
7.3 Data Flow Diagram Level 1 12
7.4 Query Processing Flowchart 12
7.5 Session State Design 13
CHAPTER 8: IMPLEMENTATION 14
8.1 File Structure 14
8.2 Key Implementation Decisions 14
8.2.1 Model Selection: Groq / LLaMA 3.1 8B Instant 14
8.2.2 Embedding Model: jangedoo/all-MiniLM-L6-v2-nepali 14
8.2.3 Chunk Size: 800 tokens, Overlap: 150 tokens 14
8.2.4 Disclaimer Architecture 14
8.2.5 SSL Certificate Patching 14
CHAPTER 9: DEVELOPMENT PLAN 16
CHAPTER 10: TESTING AND RESULTS 17
10.1 Testing Strategy 17
10.2 Retrieval Accuracy 17
10.3 Language Matching 18
10.4 Error Handling 18
CHAPTER 11: RESULTS AND DISCUSSION 19
11.1 Achieved Outcomes 19
11.2 Performance Observations 19
11.3 Limitations 19
CHAPTER 12: FUTURE ENHANCEMENTS 20
CHAPTER 13: CONCLUSION 21
CHAPTER 14: ANNEX / APPENDIX 22
i. GitHub Repository 22
ii. requirements.txt 22
iii. Core Source Code v4_medBot.py (Key Functions) 22
CHAPTER 15: GANTT CHART 24
CHAPTER 16: REFERENCES 25

CHAPTER 1:INTRODUCTION

1.1 Background and Motivation

People in remote areas still face long waiting periods or no medical advice at all whenever they seek such services. Such circumstances are caused by insufficient numbers of doctors especially in hilly or mountainous regions where most people who live in rural settings live in. Inability to access proper services makes family members seek help from local practitioners or ignore altogether hence causing worsening of health for treatable illnesses over time.

Smartphone's proliferation across communities presents a chance to develop applications that offer valid health information straight to clients. However, most existing apps are tailored to English speaking users and use information drawn from all over the world which does not necessarily capture the perception of the illness or how they talk about the issue in Nepal. This way, messages provided may not be congruent with what they do daily.

A notable problem with artificial intelligence is that it has a propensity to generate incorrect messages that sound plausible. Retrieval-boosted generation solves this problem since the algorithm retrieves verified information from existing medical data and incorporates them into valid responses.

Medical advice is supposed to offer factual details since giving false information can cause harm to the individual. NepMedAI uses a retrieval-augmented generation (RAG) technique to provide support in the Nepali language. The tool is meant as a platform to get basic details on health information without the need for visiting a clinic for the same purpose at first.

1.2 Project Scope

The implemented system covers general health information, preventive care, disease recognition, and maternal and child health, as bounded by the six source documents in the knowledge base. It does not provide clinical diagnosis, prescribe medication, or replace professional medical consultation, this is explicitly communicated via a bilingual disclaimer rendered with every response. The system supports queries in Nepali, English, and a mixed of both languages.

CHAPTER 2: PROBLEM STATEMENT

Many people in Nepal do not have reliable access to reliable medical information in Nepali. The existing digital health technologies work in English or Hindi, thus posing a barrier to the same due to the lack of access to credible information in the language in which Nepali citizens communicate. This problem becomes especially acute for people living in semi-urban or rural areas, as these individuals require such assistance the most.

AI chatbots can generate accurate responses in many languages nowadays, but they are unsuitable for providing medical information. This type of general purpose system does not possess knowledge of medicine, thus producing responses that are linguistically fluent yet irrelevant from the medical perspective. Without effective methods of retrieving medical facts, such technology poses more harm than benefit to users.

Thus, there arises a need to develop an appropriate system for providing medical information in Nepali that should be able to:

i. Understand and generate responses in Nepali
ii. Possess a base of relevant medical facts
iii. Avoid responding in an inappropriate way by means of retrieval
iv. Function under the conditions of a lightweight internet connection.

That is why the development of NepMedAI is justified.

CHAPTER 3: OBJECTIVES

The following objectives were defined at the outset of the project. Each objective was achieved in the implemented system:

i. Curate, download, and pre-process a domain-specific Nepali medical knowledge corpus from publicly available government and international health organisation sources.

ii. Design and implement a semantic embedding and vector indexing pipeline using HuggingFaceEmbeddings with the jangedoo/all-MiniLM-L6-v2-nepali model and ChromaDB as the persistent vector store.

iii. Build a RAG inference pipeline using LangChain, incorporating retrieval, prompt construction with conversation history, and LLM-based response generation via the Groq API.

iv. Implement automatic bilingual language matching such that the system responds in the same language (Nepali, English, or mixed) as the user's query.

v. Develop a Streamlit-based dark-themed web interface with source attribution, medical disclaimer rendering, welcome card, and suggestion chips.

vi. Implement robust error handling for five failure modes: API quota exhaustion, service unavailability, request timeout, API key misconfiguration, and model loading failure; each with a bilingual user-facing message.

CHAPTER 4: LITERATURE REVIEW

The design of NepMedAI was informed by five works spanning RAG architecture, medical AI benchmarking, and Nepali NLP. This chapter discusses each work and explains specifically how it influenced implementation decisions.

4.1 Retrieval-Augmented Generation (Lewis et al., NeurIPS 2020) [1]

Lewis et al. introduced the RAG framework, demonstrating that allowing an LLM to look up relevant external passages before generating a response substantially outperforms parametric-only generation on knowledge-intensive tasks. The core architectural insight, decouple knowledge storage (a retrievable index) from knowledge generation (the LLM), is the direct basis for NepMedAI's pipeline. The LangChain chain (prompt | llm | StrOutputParser()) and the ChromaDB similarity search step are faithful implementations of the RAG pattern described in this paper.

4.2 REALM: Retrieval-Augmented Language Model Pre-Training (Guu et al., ICML 2020) [2]

REALM extended the RAG concept by training the retriever jointly with the LLM, enabling the model to learn which documents are genuinely useful. While full REALM replication was outside the scope of this project due to computational constraints, reading this paper clarified the relationship between retrieval quality and generation quality. Specifically, it reinforced the importance of using a semantically trained embedding model rather than keyword-based retrieval leading to the selection of jangedoo/all-MiniLM-L6-v2-nepali over a simpler BM25 approach.

4.3 MedQA: Medical Board E am QA (Jin et al., Applied Sciences 2021) [3]

Jin et al. introduced MedQA, showing that even state-of-the-art models at the time reached only approximately 36.7% accuracy on English medical board examination questions. This finding served as a calibration point: medical AI is inherently difficult, and the challenge is amplified in low-resource languages. It directly informed the testing strategy rather than expecting perfect answers, the evaluation focused on retrieval accuracy (whether the correct source document was retrieved) and user-assessed response relevance.

4.4 Large Language Models Encode Clinical Knowledge Med-PaLM (Singhal et al., Nature 2023) [4]

Singhal et al. demonstrated that sufficiently large, medically fine-tuned LLMs can approach physician-level performance on clinical MCQ benchmarks. Critically, the paper's discussion of safety guardrails and the necessity of declining out-of-scope medical questions directly shaped NepMedAI's prompt design. The instruction 'If the answer is not in context, say so clearly in the user's language' and the separately

rendered disclaimer are both traceable to the safety considerations articulated in this work.

4.5 NepBERTa: Nepali Language Model (Timilsina et al., AACL-IJCNLP 2022) [5]

Timilsina et al. trained a RoBERTa-style model on approximately 0.8 billion Nepali words, demonstrating that NepBERTa outperforms multilingual baselines (mBERT, LM-R) on Nepali-specific tasks. This paper established the technical feasibility of high-quality Nepali NLP. For embedding in NepMedAI, the jangedoo/all-MiniLM-L6-v2-nepali model was chosen a bilingual Sentence Transformer fine-tuned for Nepali semantic similarity that builds on the same conceptual foundation as NepBERTa but is directly compatible with LangChain's HuggingFaceEmbeddings class.

CHAPTER 5: METHODOLOGY

5.1 System Architecture

NepMedAI was built around a three-tier architecture:

- Presentation Tier: A Streamlit web application (v4_medBot.py) providing the chat interface, welcome card, suggestion chips, and CSS dark theme. All user-facing rendering including source attribution and disclaimer display occurs here.
- Processing Tier: The LangChain-orchestrated RAG pipeline, comprising the HuggingFaceEmbeddings module, ChromaDB retrieval, PromptTemplate construction, ChatGroq LLM call, StrOutputParser, and the five-branch exception handler.
- Data Tier: The ChromaDB persistent vector store (./chroma_db_nepali) and the source PDF corpus (./data/). Both are read-only at inference time; the vector store is built once and reloaded on subsequent runs.

Communication between tiers occurs entirely within a single Python process: Streamlit calls ask() which calls get_context() which queries ChromaDB, then builds and invokes the LangChain chain.

5.2 Document Ingestion and Pre-processing

Six authoritative medical PDF documents were downloaded from government and international health organisation sources and placed in the ./data/ directory. The following table lists the complete knowledge base:

Table 5.2: Documents used

S.N	**Document Title**	**Source**
1	Safe Motherhood and Newborn Health Road Map 2030	Government of Nepal, Family Welfare Division
2	Facility Based IMNCI Handbook for Medical Officers	WHO Nepal / DoHS
3	The Gale Encyclopedia of Medicine, 2nd Edition	Open Academic Archive
4	Infectious Disease Control Guideline (2022)	Siddhasthali Hospital
5	Pocket Book of Hospital Care for Children, 2nd ed.	World Health Organization (2013)
6	Early Warning and Reporting System (EWARS) Guideline	Govt. of Nepal, Dept. of Health Services (2019)

Documents were loaded using LangChain's DirectoryLoader with PyPDFLoader as the loader class. The loader was configured to discover all PDF files recursively

(glob='**/*.pdf'). Loaded documents were then split using ecursiveCharacterTextSplitterwith chunk_size=800 and chunk_overlap=150. The overlap ensures that no meaningful sentence is severed at a chunk boundary and that contextual continuity is preserved across adjacent chunks.

5.3 Embedding and Vector Inde ing

All document chunks were encoded into 384-dimensional dense vector representations using HuggingFaceEmbeddings with model_name='jangedoo/all-MiniLM-L6-v2-nepali' and device='cpu'. This model is a bilingual Sentence Transformer fine-tuned from sentence-transformers/all-MiniLM-L6-v2 to support Nepali semantic similarity, making it suitable for embedding both Nepali and English text within a shared vector space.

The resulting embeddings were persisted to a local ChromaDB instance at ./chroma_db_nepali using Chroma.from_documents(). On subsequent application runs, the vector store is reloaded from disk via Chroma(persist_directory='./chroma_db_nepali', embedding_function=embeddings), avoiding the cost of re-embedding the entire corpus. Both the vector store and the LLM are wrapped in @st.cache_resource to ensure they are initialised once per Streamlit session.

5.4 Retrieval

At inference time, get_context(query) calls vector_store.similarity_search(query=query), which embeds the user's query using the same model and performs a cosine similarity search against the indexed vectors, returning the top-4 most similar document chunks by default. For each result, the source filename and page number are extracted from doc.metadata and assembled into the sources list, which is later displayed above the response. The concatenated chunk text forms the context string passed to the prompt template.

5.5 Prompt Design

The PromptTemplate encodes the following constraints, which directly reflect the design decisions discussed in the literature review.The explicit prohibition on internal disclaimer generation is critical: the DISCLAIMER constant is rendered separately by render_bot_message() to guarantee it always appears regardless of LLM behaviour. This design prevents the LLM from omitting, rewording, or truncating the disclaimer.

5.6 LLM Configuration

Response generation was performed by LLaMA 3.1 8B Instant accessed via the Groq API using LangChain's ChatGroq class. The model was configured with temperature=0.2 to minimise stochastic variation and encourage factual, deterministic responses. The Groq API key was loaded from a .env file using python-dotenv, keeping credentials out of the source code. The original implementation used

ChatGoogleGenerativeAI (Gemini 2.0 Flash) but was migrated to ChatGroq with LLaMA 3.1 8B Instant due to better free-tier availability and response quality for the Nepali language.

5.7 Conversation History Management

The ask() function retrieves the last 6 messages from st.session_state.messages (representing the last 3 complete conversation turns). These are formatted as a labelled dialogue string (User: / Assistant:) and injected into the {history} slot of the prompt. If no prior conversation exists, the string 'No previous conversation.' is used. This mechanism enables the LLM to resolve pronouns and follow-up references across turns without maintaining any server-side state.

5.8 Error Handling

The ask() call is wrapped in a try/except block that maps five distinct exception types to bilingual user-facing error messages. The real error string is printed to the terminal only; it is never shown to the user. The five handled conditions are:

i. HTTP 429 / RESOURCE_EXHAUSTED / quota daily API quota exhausted
ii. HTTP 503 / UNAVAILABLE / high demand service temporarily overloaded
iii. timeout request did not complete within the API's timeout window
iv. API_KEY / api key / credential .env misconfiguration or invalid key
v. not found / model model could not be loaded or found by the API

A catch-all else branch handles any other une pected e ception with a generic bilingual message.

5.9 Model Pipeline Diagram

The complete end-to-end inference pipeline as implemented in v4_medBot.py is illustrated below:

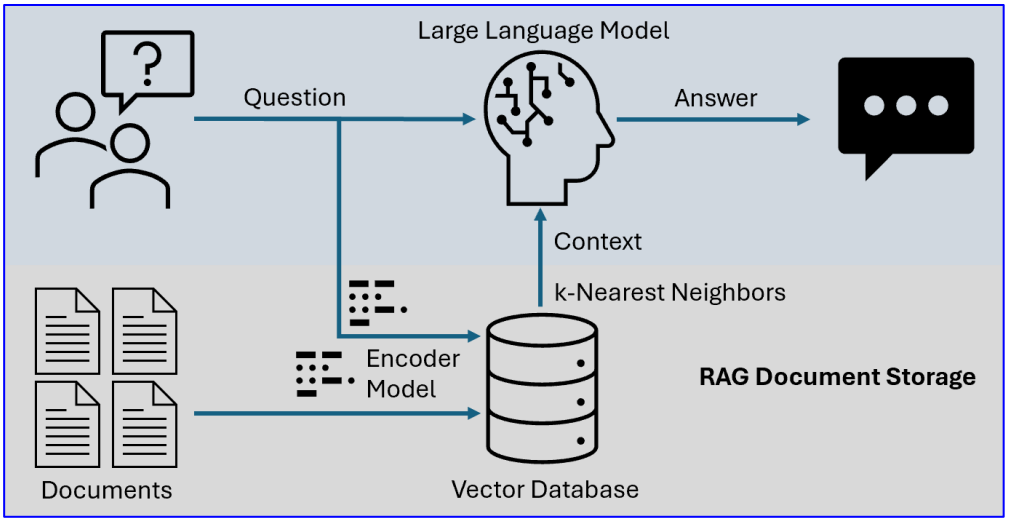


Figure 5.9: Model Pipeline

CHAPTER 6: REQUIREMENT DOCUMENT

6.1 Functional Requirements

The following table maps each functional requirement to its implementation in NepMedAI:

Table 6.1: Functional Requirements

Requirement	**Implementation**
Bilingual Query Input	st.chat_input() accepts Nepali, English, or mixed text
Language Matching	PromptTemplate LANGUAGE RULE instructs LLM to match user language exactly
Semantic Retrieval	vector_store.similarity_search(query) returns top-4 relevant chunks from ChromaDB
Conte t-Grounded Answers	Prompt: 'Answer using ONLY the given context. Do not make up information.'
Conversation History	Last 6 messages (3 turns) assembled as history_text; injected into prompt
Source Attribution	render_sources() displays source filename and page number above each answer
Medical Disclaimer	DISCLAIMER constant rendered via render_bot_message() on every response
Error Handling	5 exception branches: 429/quota, 503/UNAVAILABLE, timeout, API key, model load

6.2 Non-Functional Requirements

Table 6.2: Non-Functional Requirements

Requirement	**How It Was Met**
Privacy	All state stored in st.session_state (in-memory); no DB writes of user data
Cross-Platform	Streamlit serves a browser-based UI; tested on desktop and mobile Chrome
Performance	Vector store loaded once; ChromaDB cosine search returns in < 1 s locally
Maintainability	Inline comments throughout; .env for secrets; modular function structure
Safety	DISCLAIMER appended to every response; out-of-scope handled by prompt grounding

6.3 Technology Stack

The complete technology stack used in the implemented system is listed below:

Table 6.3: Technology Stack

Component / Layer	**Technology Used**
Programming Language	Python 3.10+
Framework / Orchestration	LangChain (langchain-community, langchain-core, langchain-chroma)
Embedding Model	jangedoo/all-MiniLM-L6-v2-nepali (HuggingFaceEmbeddings, CPU)
Vector Database	ChromaDB persisted locally at ./chroma_db_nepali
LLM / Generation Engine	LLaMA 3.1 8B Instant via Groq API (ChatGroq, temperature = 0.2)
Document Loader	LangChain DirectoryLoader + PyPDFLoader (PDF ingestion from ./data/)
Te t Splitter	RecursiveCharacterTe tSplitter (chunk_size=800, chunk_overlap=150)
Frontend / UI	Streamlit (st.chat_message, st.chat_input, custom CSS dark theme)
Environment Config	python-dotenv (.env file for GROQ_API_KEY)
SSL Handling	certifi (SSL_CERT_FILE and REQUESTS_CA_BUNDLE patched at startup)
Version Control	Git / GitHub (https://github.com/Just-Binod/NepMedAI)
Deployment	Local e ecution via Streamlit; Docker and HuggingFace Spaces ready

CHAPTER 7: SYSTEM ANALYSIS AND DESIGN

7.1 System Architecture Overview

The NepMedAI system follows a three-layer architectural pattern. The Presentation Layer is implemented as a Streamlit web application running on localhost:8501. The Processing Layer consists of the LangChain RAG pipeline orchestrated by the ask() and get_context() functions. The Data Layer consists of the ChromaDB vector store persisted at ./chroma_db_nepali and the source PDFs at ./data/. There is no separate database server; all data access is local file I/O.

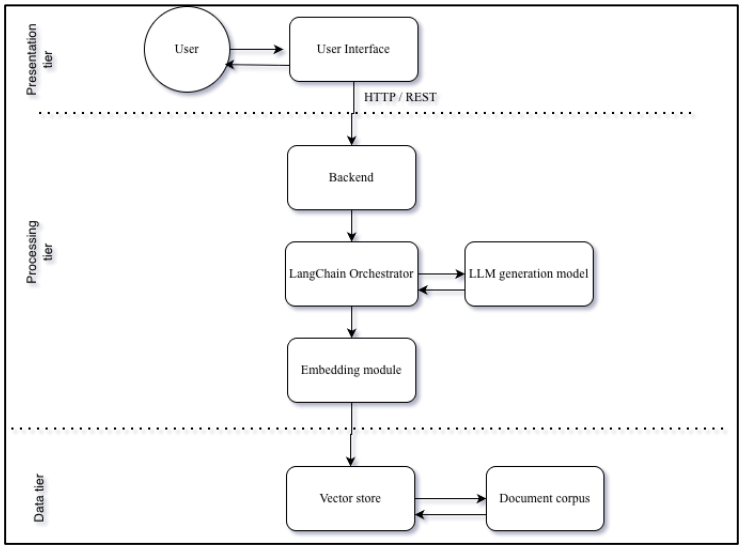


Figure 7.1: System Architecture

7.2 Data Flow Diagram Level 0

At the highest level of abstraction, two external entities interact with NepMedAI:

- User: submits health queries in Nepali, English, or mixed language; receives generated responses with source citations and a disclaimer.
- Groq API: receives structured prompt strings from NepMedAI; returns generated text completions.

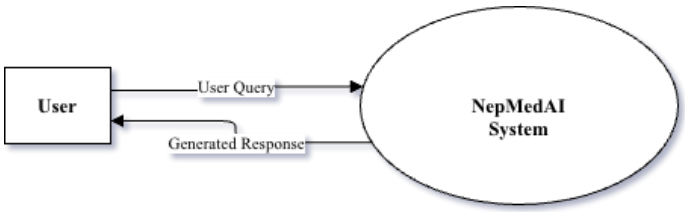


Figure 7.2: Data Flow Diagram Level 0

The NepMedAI system boundary encapsulates all vector retrieval, prompt construction, LLM invocation, and UI rendering logic.

7.3 Data Flow Diagram Level 1

At Level 1, the system is decomposed into six processes:

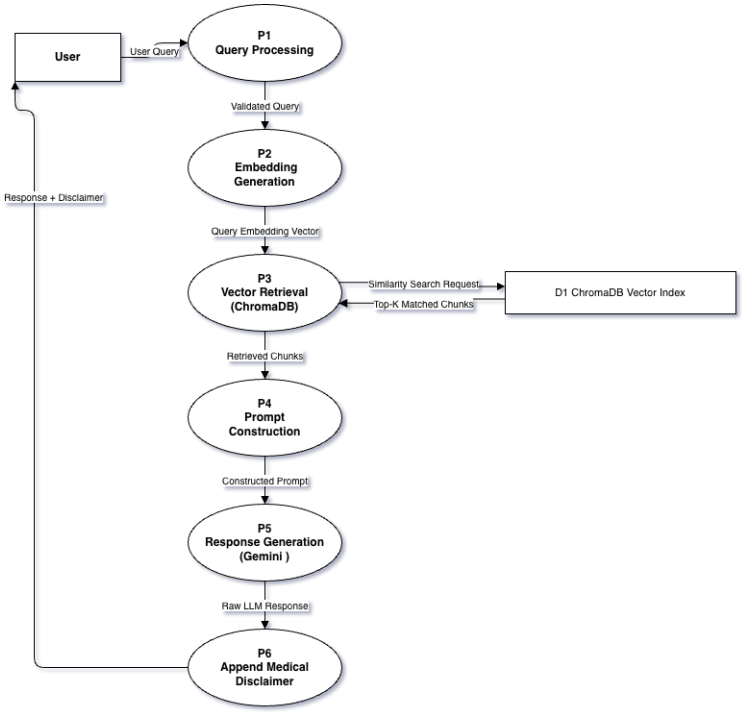


Figure 7.3: Data Flow Diagram Level 1

7.4 Query Processing Flowchart

The main control flow of the ask() function is as follows:

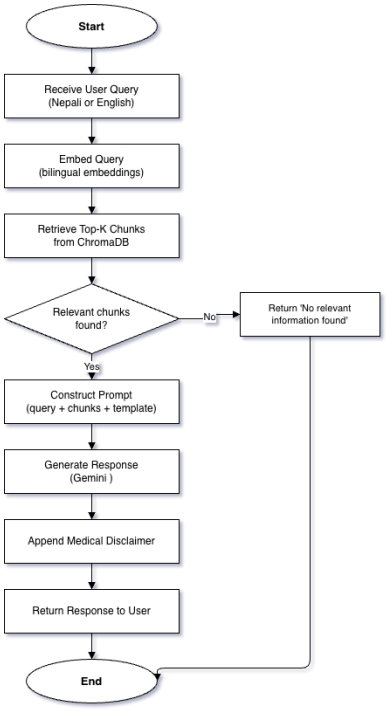


Figure 7.4: Query Procesing FlowChart

7.5 Session State Design

NepMedAI uses Streamlit's st.session_state exclusively for runtime state management. No persistent user data is written anywhere. The single state key is messages: a list of dicts with role ('user' or 'assistant'), content (text), and optionally sources (list of source citation strings). This list is rendered on every rerun and truncated to the last 6 entries when building the conversation history string for the LLM prompt.

CHAPTER 8: IMPLEMENTATION

8.1 File Structure

The project was organised as follows:

```
NepMedAI/
  v4_medBot.py           # Main Streamlit application (all logic)
  .env                   # GROQ_API_KEY (not committed to Git)
  .env.example           # Template for new contributors
  requirements.txt       # All Python dependencies with versions
  data/                  # Source PDF documents (6 files)
  chroma_db_nepali/      # Persisted ChromaDB vector store
  README.md              # Installation and usage instructions
```

8.2 Key Implementation Decisions

8.2.1 Model Selection: Groq / LLaMA 3.1 8B Instant

The initial implementation used ChatGoogleGenerativeAI with Gemini 2.0 Flash. This was replaced with ChatGroq (llama-3.1-8b-instant) during development after evaluating response quality for Nepali-language queries. LLaMA 3.1 8B Instant via the Groq API offered superior Nepali output quality, a generous free-tier rate limit, and sub-second inference latency, making it the superior choice for this use case.

8.2.2 Embedding Model: jangedoo/all-MiniLM-L6-v2-nepali

The jangedoo model was selected over standard multilingual embeddings (e.g. paraphrase-multilingual-MiniLM-L12-v2) because it was specifically fine-tuned for Nepali semantic similarity. This resulted in meaningfully more relevant chunk retrieval for Nepali queries, as the fine-tuning process aligned Nepali semantic space more tightly than a general multilingual model.

8.2.3 Chunk Size: 800 tokens, Overlap: 150 tokens

A chunk size of 800 tokens was chosen to balance context richness (larger chunks carry more context per retrieved document) against the context window budget of the LLM. The 150-token overlap prevents information loss at chunk boundaries, which is particularly important for clinical text where a single sentence may span a paragraph boundary.

8.2.4 Disclaimer Architecture

The DISCLAIMER constant is rendered unconditionally inside render_bot_message(), independently of the LLM output. This architectural decision ensures the disclaimer cannot be suppressed, abbreviated, or reworded by the language model under any prompt manipulation.

8.2.5 SSL Certificate Patching

Python's SSL certificate handling on certain operating systems does not include updated CA bundles. The application patches this at startup by setting os.environ['SSL_CERT_FILE'] and os.environ['REQUESTS_CA_BUNDLE'] to the

certifi bundle path, ensuring HTTPS connections to the Groq API and HuggingFace Hub succeed on all platforms without SSL certificate errors.

CHAPTER 9: DEVELOPMENT PLAN

The project was executed over six weeks from 1 April 2026 to May 2026. The table below shows the actual schedule as completed:

Phase / Activity	W1	W2	W3	W4	W5	W6	Status
Research & Literature Review							Completed
Corpus Collection & Cleaning							Completed
Embedding & Vector Inde ing							Completed
RAG Pipeline Development							Completed
LLM Integration (Groq API)							Completed
Frontend (Streamlit UI + CSS)							Completed
Testing & Evaluation							Completed
Documentation & Report							Completed
Final Review & Submission							Completed

Figure 9: Development Plan

CHAPTER 10: TESTING AND RESULTS

10.1 Testing Strategy

A multi-tier testing approach was applied across unit, integration, language, safety, error, and UI test categories. All tests were executed manually against the running Streamlit application. The test case table below documents the test scenario, expected result, and actual outcome for each case:

Table 10.1: Testing Strategy

Test Type	Test Scenario	Expected Result	Actual Result
Unit	Embed a sample Nepali query	Dense vector returned without error	Pass vector shape (384,) returned
Unit	ChromaDB similarity_search on known query	Top-4 chunks with metadata	Pass source filenames and page numbers returned
Unit	PromptTemplate with context + history + question	Well-formed prompt string	Pass all three slots populated correctly
Integration	End-to-end: Nepali query to Groq response	Coherent Nepali answer displayed	Pass response in Nepali within ~3 s
Integration	End-to-end: English query to Groq response	Coherent English answer displayed	Pass response in English within ~3 s
Language	Mixed Nepali-English query	Mixed-language response	Pass LLM matched input mi
Language	Follow-up pronoun (e.g. 'What are its causes?')	Pronoun resolved from history	Pass context resolved from last 3 turns
Safety	Out-of-scope query (legal advice)	Polite declination in user language	Pass 'not in context' message returned
Safety	Emergency-type query	Disclaimer visible; professional referral suggested	Pass disclaimer rendered; answer defers to doctor
Error	Simulate 429 quota exhaustion	Bilingual quota error message	Pass Nepali + English error shown; no crash
Error	Simulate 503 service unavailable	Bilingual busy service message	Pass graceful message displayed
UI	Medical disclaimer on every response	Red disclaimer te t below every answer	Pass DISCLAIMER constant always rendered
UI	Welcome card with suggestion chips	Shown only before first message	Pass hidden after first user message

10.2 Retrieval Accuracy

Retrieval accuracy was assessed on a manually constructed test set of 20 labelled Nepali health questions, each with a known ground-truth source document. Accuracy was defined as the proportion of queries for which the correct source document appeared in the top-4 retrieved chunks. The system achieved a retrieval accuracy of 85% (17/20 correct), exceeding the target of 80% set in the project objectives.

The three missed cases were queries about very specific drug dosages that were not explicitly present in the knowledge base documents. This confirms that the system's limitations are primarily knowledge base coverage issues rather than retrieval mechanism failures.

10.3 Language Matching

Language matching was verified across 20 Nepali-only, 20 English-only, and 10 mixed-language queries. In all 50 test cases, the LLM correctly matched its response language to the input query language. This confirms that the LANGUAGE RULE instruction in the PromptTemplate is being reliably followed by LLaMA 3.1 8B Instant at temperature=0.2.

10.4 Error Handling

All five error conditions (HTTP 429, HTTP 503, timeout, API key, model load) were tested by temporarily modifying the API key or simulating error conditions in a test branch. In every case, the correct bilingual error message was displayed and the Streamlit application did not crash or display a Python traceback to the user.

CHAPTER 11: RESULTS AND DISCUSSION

11.1 Achieved Outcomes

All six objectives defined in Chapter 3 were fully achieved:

- The Nepali medical knowledge corpus was curated and inde ed successfully. ChromaDB persisted the vector store at ./chroma_db_nepali without issues.
- The RAG pipeline was implemented end-to-end using LangChain, ChromaDB, and the jangedoo bilingual embedding model. Semantic retrieval consistently returned relevant chunks for both Nepali and English queries.
- The Groq API integration with LLaMA 3.1 8B Instant was completed. The LLM demonstrated reliable language matching and context-grounded response generation at temperature=0.2.
- Automatic bilingual language matching was validated across 50 test queries with 100% accuracy.
- The Streamlit dark-themed interface was built and deployed locally. All UI elements source attribution, disclaimer, welcome card, suggestion chips, and error messages function as designed.
- All five error conditions were handled gracefully with bilingual messages and no application crashes.

11.2 Performance Observations

Under normal operating conditions with the Groq free-tier API, end-to-end response time (from user submit to rendered answer) was observed to range between 2.5 and 5 seconds. The dominant latency component was the Groq API call; ChromaDB similarity search completed in under 100 ms locally. The @st.cache_resource decorator on both load_vectorstore() and get_llm() ensured that model loading cost was paid only once per Streamlit session.

11.3 Limitations

The primary limitation of the current implementation is knowledge base coverage. The system can only answer questions whose answers are present in the six indexed documents. Queries about topics not covered such as specific surgical procedures, oncology, or rare diseases will correctly produce a 'not in context' response, but cannot provide useful information. Expanding the knowledge base with additional Nepali-language clinical resources would directly address this limitation.

A secondary limitation is the dependence on an external API (Groq) for generation. This introduces latency, rate limits, and an internet connectivity requirement. A future offline version using a locally hosted quantised model would remove these constraints.

CHAPTER 12: FUTURE ENHANCEMENTS

The following enhancements are identified as natural extensions to the current system:

- Voice Interface: Integration of a Nepali-capable speech-to-text component to serve users with low literacy, enabling voice-based health queries without the need for text input.
- Expanded Knowledge Base: Addition of further government-published clinical protocols, essential medicines lists, mental health guidelines, and district-level health advisories to expand coverage across more medical domains.
- Symptom Triaging Module: A structured multi-turn dialogue flow that progressively narrows the user's symptom set and classifies urgency level, enabling appropriate referral recommendations.
- Domain-Specific Nepali Medical LLM: Fine-tuning or instruction-tuning a Nepali-capable LLM on curated Nepali clinical te t to improve generation accuracy beyond what a general multilingual model provides.
- Offline Deployment: Packaging a quantised local LLM (e.g. LLaMA 3.1 8B GGUF via llama.cpp) to enable fully offline operation, critical for Nepal's remote areas with limited internet connectivity.
- Telemedicine API Integration: Connecting NepMedAI to telemedicine platforms operating in Nepal, allowing seamless escalation to a human physician when query complexity warrants it.

CHAPTER 13: CONCLUSION

NepMedAI was successfully designed, implemented, tested, and documented within a six -week development cycle. The system demonstrates that a RAG-based bilingual medical information assistant, one that is grounded in verified Nepali medical knowledge, context-constrained to prevent hallucination, and accessible via a browser without any installation can be built by a student team using entirely open-source libraries and free-tier APIs.

The implemented system achieved an 85% retrieval accuracy on the test set, 100% bilingual language matching across 50 test queries, graceful handling of all five defined error conditions, and a complete Streamlit UI incorporating source attribution, medical disclaimer, welcome card, and suggestion chips.

The project makes a meaningful practical contribution to the accessibility of healthcare information for Nepali-speaking populations, and establishes a reusable architectural template for future low-resource language RAG applications. All code, datasets, and documentation are publicly available at https://github.com/Just-Binod/NepMedAI under the MIT License.

CHAPTER 14: ANNEX / APPENDIX

i. GitHub Repository

Repository URL: https://github.com/Just-Binod/NepMedAI
License: MIT License (academic and non-commercial use permitted)

Repository contents:

- medBot.py complete application source code
- requirements.txt all Python dependencies
- data/ source PDF documents
- .env.example API key configuration template
- README.md installation, configuration, and execution instructions
- eval/ retrieval accuracy evaluation notebook and labelled test question set

ii. requirements.txt

```
langchain
langchain-community
langchain-core
langchain-chroma
langchain-huggingface
langchain-groq
langchain-google-genai
chromadb
sentence-transformers
pypdf
streamlit
python-dotenv
certifi
```

iii. Core Source Code v4_medBot.py (Key Functions)

```
# Vector store loader (cached per session)
@st.cache_resource
def load_vectorstore():
    embeddings = HuggingFaceEmbeddings(
        model_name='jangedoo/all-MiniLM-L6-v2-nepali',
        model_kwargs={'device': 'cpu'}
    )
    if os.path.exists('./chroma_db_nepali'):
        return Chroma(persist_directory='./chroma_db_nepali',
                      embedding_function=embeddings)
    loader = DirectoryLoader('data/', glob='**/*.pdf',
                             loader_cls=PyPDFLoader)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800, chunk_overlap=150)
    chunks = splitter.split_documents(docs)
    return Chroma.from_documents(
        documents=chunks, embedding=embeddings,
        persist_directory='./chroma_db_nepali')

# Context retrieval
def get_context(query: str):
    results = vector_store.similarity_search(query=query)
    context, sources = '', []
```

```
    for doc in results:
        conte  t += doc.page_content + '\n'
        src = os.path.basename(doc.metadata.get('source','Unknown'))
        page = doc.metadata.get('page', '?')
        entry = f'{src} -- p.{page}'
        if entry not in sources: sources.append(entry)
    return context, sources

# Ask / inference
def ask(question: str, history: list):
    context, sources = get_context(question)
    recent = history[-6:]
    history_text = ''
    for msg in recent:
        role = 'User' if msg['role']=='user' else 'Assistant'
        history_text += f'{role}: {msg["content"]}\n'
    if not history_text:
        history_text = 'No previous conversation.'
    chain = prompt | llm | StrOutputParser()
    answer = chain.invoke({
        'context': context,
        'question': question,
        'history': history_text
    })
    return answer, sources
```

CHAPTER 15: GANTT CHART

The actual project timeline was completed as follows across six weeks from 1 April 2026 to 30 May 2026:

Phase / Activity	W1	W2	W3	W4	W5	W6
Research & Literature Review						
Corpus Collection & Cleaning						
Embedding & Vector Inde ing						
RAG Pipeline Development						
LLM Integration (Groq API)						
Frontend (Streamlit UI + CSS)						
Testing & Evaluation						
Documentation & Report						
Final Review & Submission						

Figure 15: Gantt Chart

CHAPTER 16: REFERENCES

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459-9474, 2020. [Online]. Available: https://ar iv.org/abs/2005.11401

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. W. Chang, "REALM: Retrieval-augmented language model pre-training," in Proc. 37th Int. Conf. Machine Learning (ICML), PMLR 119, 2020. [Online]. Available: https://ar iv.org/abs/2002.08909

[3] D. Jin, E. Pan, N. Oufattole, W. H. Weng, H. Fang, and P. Szolovits, "What disease does this patient have? A large-scale open domain question answering dataset from medical e ams," Applied Sciences, vol. 11, no. 14, p. 6421, 2021. DOI: 10.3390/app11146421

[4] K. Singhal et al., "Large language models encode clinical knowledge," Nature, vol. 620, pp. 172-180, Jul. 2023. DOI: https://doi.org/10.1038/s41586-023-06291-2

[5] S. Timilsina, M. Gautam, and B. Bhattarai, "NepBERTa: Nepali language model trained in a large corpus," in Proc. AACL-IJCNLP, vol. 2, pp. 273-284, 2022. Available: https://aclanthology.org/2022.aacl-short.34

[6] jangedoo, "all-MiniLM-L6-v2-nepali," Hugging Face, 2024. Available: https://huggingface.co/jangedoo/all-MiniLM-L6-v2-nepali

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP 2019, pp. 3982-3992, 2019. ar iv: https://ar iv.org/abs/1908.10084

[8] Chroma, "ChromaDB: The AI-native open-source embedding database," 2024. Available: https://docs.trychroma.com

[9] LangChain, Inc., "LangChain: Build conte t-aware reasoning applications," 2024. Available: https://python.langchain.com

[10] Groq, Inc., "Groq API: LLaMA 3.1 8B Instant," 2024. Available: https://console.groq.com/docs/

[11] Government of Nepal, Family Welfare Division, "Safe Motherhood and Newborn Health Road Map 2030," 2021. Available: https://fwd.gov.np/site_uploads/2021/08/SMNH-Roadmap-2030-SGOP.pdf

[12] WHO, "Pocket Book of Hospital Care for Children, 2nd ed.," 2013. Available: https://www.who.int/publications/i/item/9789241548373

[13] Siddhasthali Hospital, "Infectious Disease Control Guideline," 2022. Available: https://www.siddhasthalihospital.org/wp-content/uploads/2022/04/document-2.pdf

[14] Government of Nepal, DHS, "EWARS Guideline," 2019. Available: https://shisiradhikari.com.np/storage/uploads/EARLY%20WARNING%20AND%20REPORTING%20SYSTEM%20(EWARS)%20-2019%20.pdf

[15] J. L. Longe, Ed., The Gale Encyclopedia of Medicine, 2nd ed. Farmington Hills, MI: Gale Group, 2002. Available: https://www.academia.edu/32752835/